

Tensor Parallelism Backward Propagation - Gradient Communication

This document explains how gradients are communicated in VANILLA Tensor Parallelism (without Sequence Parallelism) during backward propagation.

SETUP: Forward Pass Recap

Dimensions:

- Batch (b) = 1, Sequence (s) = 4, Hidden (h) = 4
- 2 GPUs (GPU0 and GPU1)

In vanilla TP, the INPUT X is REPLICATED on all GPUs (no sequence split).

Forward pass:

$X \rightarrow [\text{Column-Linear: } Y \times A] \rightarrow \text{GeLU} \rightarrow [\text{Row-Linear: } Z \times B] \rightarrow \text{Output}$

Weight distribution:

- A (column-linear): $A = [A1 \mid A2]$, split along columns (output dim)
- B (row-linear): $B = [B1; B2]$, split along rows (input dim)

SECTION 1: FORWARD PASS WITH VALUES (Recap)

Input X (SAME on both GPUs):

```
[[[0.1, 0.2, 0.3, 0.4],
  [0.5, 0.6, 0.7, 0.8],
  [0.9, 1.0, 1.1, 1.2],
  [1.3, 1.4, 1.5, 1.6]]]
Shape: (1, 4, 4)
```

Column-Linear weights:

A1 (GPU0): (4, 2) A2 (GPU1): (4, 2)
Full A = [A1 | A2]: (4, 4)

Forward Column-Linear:

GPU0: $Z1 = \text{GeLU}(X \times A1) \rightarrow (1, 4, 2)$ [columns 0-1]
GPU1: $Z2 = \text{GeLU}(X \times A2) \rightarrow (1, 4, 2)$ [columns 2-3]

Row-Linear weights:

B1 (GPU0): (2, 4) B2 (GPU1): (2, 4)
Full B = [B1; B2]: (4, 4)

Forward Row-Linear:

GPU0: $Y1 = Z1 \times B1 \rightarrow (1, 4, 4)$ [PARTIAL]
GPU1: $Y2 = Z2 \times B2 \rightarrow (1, 4, 4)$ [PARTIAL]

Output Y = Y1 + Y2 (need ALL-REDUCE in forward!)

VANILLA TP FORWARD:

Column-Linear: No communication (independent computation)
Row-Linear: ALL-REDUCE to sum partial outputs $Y = Y1 + Y2$

SECTION 2: BACKWARD PASS OVERVIEW

Backward propagation computes gradients in REVERSE order:

dL/dY (given) $\rightarrow dL/dB$, $dL/dZ \rightarrow dL/dA$, dL/dX

Key question: When do we need ALL-REDUCE vs REDUCE-SCATTER?

ANSWER: In VANILLA TP (no SP), we use ALL-REDUCE for gradients

Why? Because input X is REPLICATED, so dL/dX must also be REPLICATED (same on all GPUs) for correct gradient accumulation.

SECTION 3: ROW-LINEAR BACKWARD (First in backward order)

Forward was: $Y = Z \times B$ where $Z = [Z1 \mid Z2]$, $B = [B1; B2]$
 $Y = Z1 \times B1 + Z2 \times B2$ (partial sums)

Given: dL/dY (gradient from upstream), shape (1, 4, 4)
 dL/dY is SAME on all GPUs (after forward all-reduce, Y is same)

ROW-LINEAR BACKWARD FORMULAS

Weight gradient: $dL/dB = Z^T \times dL/dY$
Input gradient: $dL/dZ = dL/dY \times B^T$

Step 3.1: Weight Gradient dL/dB

Each GPU computes gradient for its local weight:

GPU0: $dL/dB1 = Z1^T \times dL/dY$ Shape: (2, 4)
GPU1: $dL/dB2 = Z2^T \times dL/dY$ Shape: (2, 4)

NO COMMUNICATION NEEDED for weight gradients!
Each GPU only needs gradient for its own weights.

Step 3.2: Input Gradient dL/dZ

Need: $dL/dZ = dL/dY \times B^T$

Since $B = [B1; B2]$ stacked vertically:
 $B^T = [B1^T \mid B2^T]$ (concatenated horizontally)

Full computation:
 $dL/dZ = dL/dY \times [B1^T \mid B2^T]$
 $= [dL/dY \times B1^T \mid dL/dY \times B2^T]$
 $= [dL/dZ1 \mid dL/dZ2]$

Each GPU computes its portion:
GPU0: $dL/dZ1 = dL/dY \times B1^T$ Shape: (1, 4, 2) [gradients for Z1]
GPU1: $dL/dZ2 = dL/dY \times B2^T$ Shape: (1, 4, 2) [gradients for Z2]

NO COMMUNICATION NEEDED here either!
Why? Because Z1 and Z2 are separate tensors on different GPUs.
Each GPU needs gradient only for its own Z portion.

=====

SECTION 4: GELU BACKWARD (Element-wise, No Communication)

=====

Forward: $Z = \text{GeLU}(H)$ where $H = X \times A$

Backward: $dL/dH = dL/dZ \odot \text{GeLU}'(H)$

Each GPU applies element-wise independently:
GPU0: $dL/dH1 = dL/dZ1 \odot \text{GeLU}'(H1)$ Shape: (1, 4, 2)
GPU1: $dL/dH2 = dL/dZ2 \odot \text{GeLU}'(H2)$ Shape: (1, 4, 2)

NO COMMUNICATION NEEDED.

=====

SECTION 5: COLUMN-LINEAR BACKWARD (This is where ALL-REDUCE happens!)

=====

Forward was: $H = X \times A$ where $A = [A1 \mid A2]$
 $H1 = X \times A1$ (GPU0), $H2 = X \times A2$ (GPU1)

Given: $dL/dH1$ on GPU0, $dL/dH2$ on GPU1 (each (1, 4, 2))

COLUMN-LINEAR BACKWARD FORMULAS	
Weight gradient:	$dL/dA = X^T \times dL/dH$
Input gradient:	$dL/dX = dL/dH \times A^T$

Step 5.1: Weight Gradient dL/dA

Each GPU computes gradient for its local weight:

```
GPU0: dL/dA1 = X^T × dL/dH1      Shape: (4, 2)
GPU1: dL/dA2 = X^T × dL/dH2      Shape: (4, 2)
```

NO COMMUNICATION NEEDED for weight gradients!

Step 5.2: Input Gradient $dL/dX \leftarrow$ HERE IS WHERE ALL-REDUCE HAPPENS!

Need: $dL/dX = dL/dH \times A^T$

Since $H = [H1 \mid H2]$ and $A = [A1 \mid A2]$:

$$\begin{aligned} dL/dX &= dL/dH \times A^T \\ &= [dL/dH1 \mid dL/dH2] \times \begin{bmatrix} A1^T \\ A2^T \end{bmatrix} \\ &= dL/dH1 \times A1^T + dL/dH2 \times A2^T \end{aligned}$$

Each GPU computes PARTIAL gradient:

```
GPU0: dL/dX_partial = dL/dH1 × A1^T      Shape: (1, 4, 4)
GPU1: dL/dX_partial = dL/dH2 × A2^T      Shape: (1, 4, 4)
```

ALL-REDUCE NEEDED!

$dL/dX_{full} = dL/dX_{partial}(GPU0) + dL/dX_{partial}(GPU1)$

After ALL-REDUCE, both GPUs have the SAME complete dL/dX

Why ALL-REDUCE (not REDUCE-SCATTER)?

In vanilla TP, X is REPLICATED on all GPUs.

Therefore, dL/dX must also be REPLICATED for correct backprop to previous layer.

If we used REDUCE-SCATTER:

- Each GPU would have partial dL/dX
- But next backward layer expects FULL dL/dX
- Would need to all-gather, defeating the purpose

=====

SECTION 6: NUMERICAL EXAMPLE - COLUMN-LINEAR BACKWARD

=====

Let's compute with actual numbers.

Given (simplified values):

X (same on both GPUs):

```
[[[1, 2, 3, 4],
   [5, 6, 7, 8]]]      Shape: (1, 2, 4)
```

$A1$ (GPU0):

```
[[1, 2],
 [5, 6],
 [9, 10],
```

$A2$ (GPU1):

```
[[3, 4],
 [7, 8],
 [11, 12],
```

```
[13, 14]]           [15, 16]]
Shape: (4, 2)       Shape: (4, 2)
```

Forward results:

```
H1 = X × A1:           H2 = X × A2:
[[[90, 100],           [[110, 120],
 [202, 228]]]          [246, 272]]]
```

Assume upstream gradient (after GeLU backward):

```
dL/dH1 (GPU0):         dL/dH2 (GPU1):
[[[0.1, 0.2],           [[0.3, 0.4],
 [0.5, 0.6]]]           [0.7, 0.8]]]
```

Step 6.1: Compute Partial dL/dX on each GPU

GPU0: $dL/dX_{\text{partial}} = dL/dH1 \times A1^T$

```
A1^T = [[1, 5, 9, 13],
        [2, 6, 10, 14]]
```

$dL/dH1 \times A1^T$:

```
Row 0: [0.1, 0.2] × A1^T = [0.1×1+0.2×2, 0.1×5+0.2×6, 0.1×9+0.2×10, 0.1×13+0.2×14]
                        = [0.5, 1.7, 2.9, 4.1]
Row 1: [0.5, 0.6] × A1^T = [0.5×1+0.6×2, 0.5×5+0.6×6, 0.5×9+0.6×10, 0.5×13+0.6×14]
                        = [1.7, 6.1, 10.5, 14.9]
```

dL/dX_{partial} (GPU0):

```
[[[0.5, 1.7, 2.9, 4.1],
 [1.7, 6.1, 10.5, 14.9]]]
```

GPU1: $dL/dX_{\text{partial}} = dL/dH2 \times A2^T$

```
A2^T = [[3, 7, 11, 15],
        [4, 8, 12, 16]]
```

$dL/dH2 \times A2^T$:

```
Row 0: [0.3, 0.4] × A2^T = [0.3×3+0.4×4, 0.3×7+0.4×8, 0.3×11+0.4×12, 0.3×15+0.4×16]
                        = [2.5, 5.3, 8.1, 10.9]
Row 1: [0.7, 0.8] × A2^T = [0.7×3+0.8×4, 0.7×7+0.8×8, 0.7×11+0.8×12, 0.7×15+0.8×16]
                        = [5.3, 11.3, 17.3, 23.3]
```

dL/dX_{partial} (GPU1):

```
[[[2.5, 5.3, 8.1, 10.9],
 [5.3, 11.3, 17.3, 23.3]]]
```

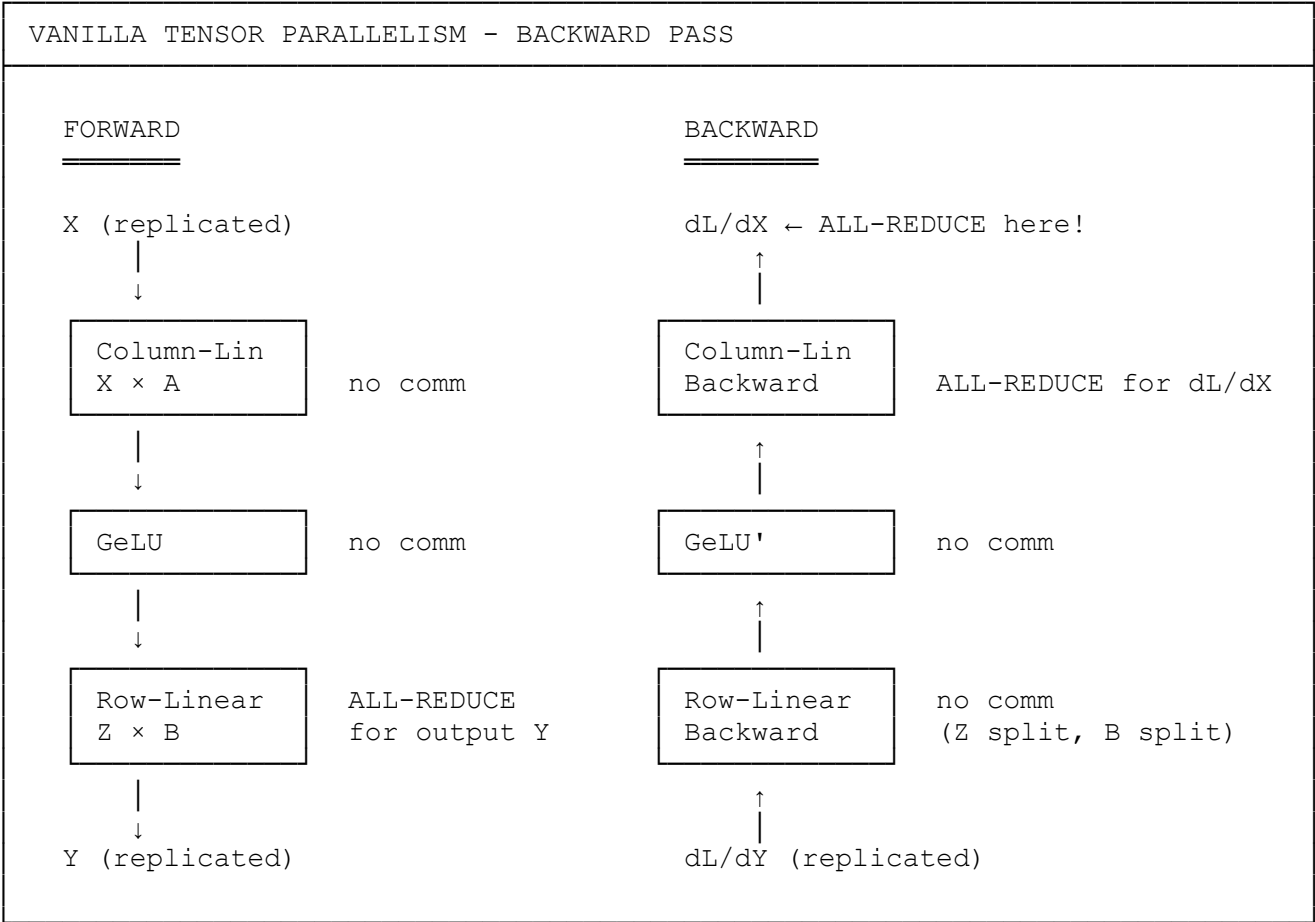
Step 6.2: ALL-REDUCE to get full dL/dX

$dL/dX = dL/dX_{\text{partial}}(\text{GPU0}) + dL/dX_{\text{partial}}(\text{GPU1})$

```
[[[0.5+2.5, 1.7+5.3, 2.9+8.1, 4.1+10.9],
 [1.7+5.3, 6.1+11.3, 10.5+17.3, 14.9+23.3]]]
```

dL/dX (SAME on both GPUs after ALL-REDUCE):
[[[3.0, 7.0, 11.0, 15.0],
[7.0, 17.4, 27.8, 38.2]]]

SECTION 7: COMPLETE BACKWARD COMMUNICATION PATTERN



Communication Summary:

Layer	Forward Comm	Backward Comm
Column-Linear	None	ALL-REDUCE (for dL/dX)
GeLU	None	None
Row-Linear	ALL-REDUCE (for Y)	None
TOTAL per MLP	1 ALL-REDUCE	1 ALL-REDUCE

SECTION 8: WHY NOT REDUCE-SCATTER IN VANILLA TP BACKWARD?

=====

Key insight: The communication pattern depends on data distribution.

VANILLA TP:

- Input X: REPLICATED (same on all GPUs)
 - Output Y: REPLICATED (after all-reduce)
 - Activations: SPLIT along hidden dimension
- dL/dX must be REPLICATED → need ALL-REDUCE

SP + TP:

- Input X*: SPLIT along sequence dimension
 - Output Y*: SPLIT along sequence dimension
 - Activations in TP region: REPLICATED
- dL/dX* can be SPLIT → can use REDUCE-SCATTER

COMPARISON: VANILLA TP vs SP+TP
VANILLA TP (this document): Forward Row-Linear: ALL-REDUCE (sum partials, replicate output) Backward Column-Linear: ALL-REDUCE (sum partials, replicate gradient) SP + TP (previous document): Forward Row-Linear: REDUCE-SCATTER (sum + distribute to SP) Backward Column-Linear: REDUCE-SCATTER (sum + distribute to SP) SP+TP saves 50% communication by using REDUCE-SCATTER instead of ALL-REDUCE

=====

SECTION 9: ALL-REDUCE IMPLEMENTATION DETAIL

=====

Ring ALL-REDUCE = REDUCE-SCATTER + ALL-GATHER

Phase 1: REDUCE-SCATTER

- GPU0 sends partial[chunk1] to GPU1, receives partial[chunk0] from GPU1
- GPU0: sum[chunk0] = partial0[chunk0] + partial1[chunk0]
- GPU1: sum[chunk1] = partial0[chunk1] + partial1[chunk1]

Phase 2: ALL-GATHER

- GPU0 sends sum[chunk0] to GPU1
- GPU1 sends sum[chunk1] to GPU0
- Both GPUs now have complete sum

Example with dL/dX:

Before ALL-REDUCE:

- GPU0: dL/dX_partial0 = [[[0.5, 1.7, 2.9, 4.1], [1.7, 6.1, 10.5, 14.9]]]
- GPU1: dL/dX_partial1 = [[[2.5, 5.3, 8.1, 10.9], [5.3, 11.3, 17.3, 23.3]]]

Chunk division (split along last dim for example):
 Chunk0: columns 0-1 Chunk1: columns 2-3

REDUCE-SCATTER phase:
 GPU0: $\text{sum}[\text{chunk0}] = [0.5+2.5, 1.7+5.3] + [1.7+5.3, 6.1+11.3] = [[3.0, 7.0], [7.0, 17.4]]$
 GPU1: $\text{sum}[\text{chunk1}] = [2.9+8.1, 4.1+10.9] + [10.5+17.3, 14.9+23.3] = [[11.0, 15.0], [27.8, 38.2]]$

ALL-GATHER phase:
 GPU0 receives chunk1 from GPU1
 GPU1 receives chunk0 from GPU0

After ALL-REDUCE (both GPUs identical):
 $dL/dX = [[[3.0, 7.0, 11.0, 15.0], [7.0, 17.4, 27.8, 38.2]]]$

=====

SECTION 10: SUMMARY - WHEN TO USE WHICH COLLECTIVE

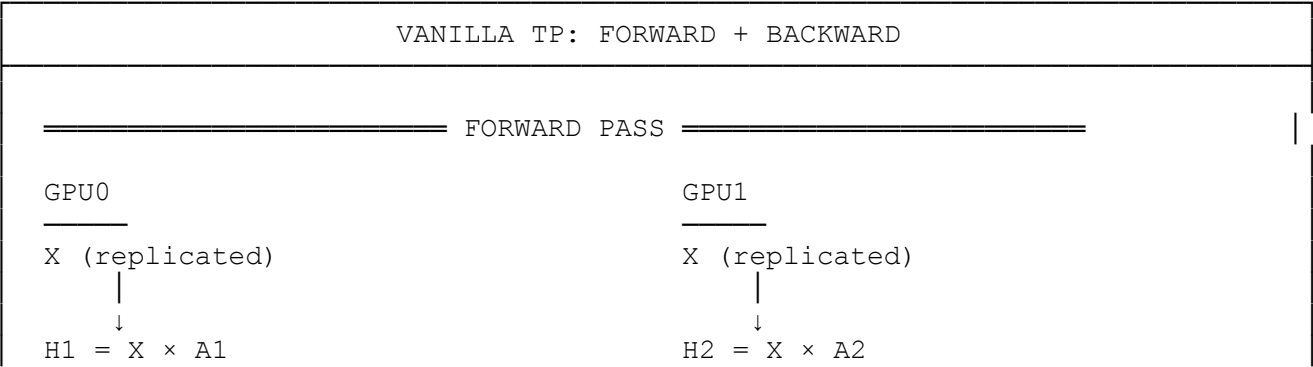
=====

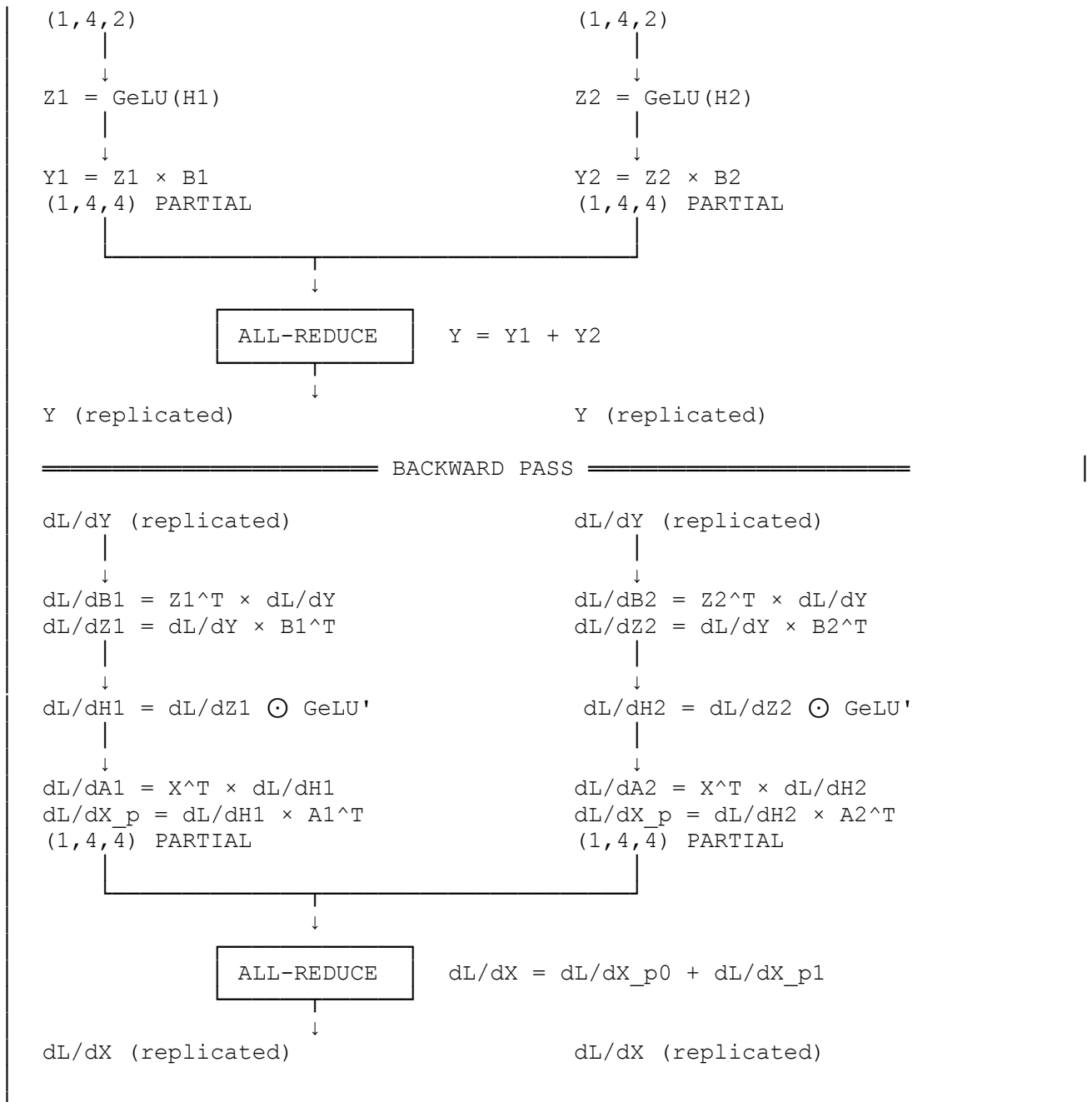
Collective	Use When
ALL-REDUCE	Need FULL result on ALL GPUs - Vanilla TP forward (row-linear output) - Vanilla TP backward (column-linear input grad) - Data parallel gradient sync
REDUCE-SCATTER	Need PARTIAL result on EACH GPU (distributed) - SP+TP transitions (TP→SP) - ZeRO optimizer state distribution
ALL-GATHER	Need to REPLICATE distributed data - SP+TP transitions (SP→TP) - Gathering split activations

=====

SECTION 11: DIAGRAM - VANILLA TP FULL FORWARD + BACKWARD

=====





=====

SECTION 12: KEY TAKEAWAYS

=====

1. VANILLA TP uses ALL-REDUCE (not reduce-scatter) in backward because:
 - Input X is replicated $\rightarrow dL/dX$ must be replicated
 - All GPUs need identical gradient to continue backprop
2. Communication locations are SYMMETRIC:
 - Forward: ALL-REDUCE after row-linear
 - Backward: ALL-REDUCE after column-linear backward

3. Weight gradients (dL/dA , dL/dB) don't need communication:

- Each GPU maintains its own weight shard
- Each GPU accumulates gradient for its shard only
- Optimizer updates are independent per shard

4. SP+TP improves on this by:

- Using REDUCE-SCATTER instead of ALL-REDUCE
- Saving 50% communication bandwidth
- Trading memory (split activations) for communication efficiency

=====
END OF DOCUMENT
=====